

深度调研与代码实现映射：让国创赛材料贴近真实用户痛点

调研日期：2026-05-25

目标：把“AI Agent 安全”从概念叙事落到用户真实痛点、代码实现证据和可演示场景。

1. 外部调研结论

1.1 Agent 安全的痛点不是“模型回答错”，而是“高权限执行链失控”

多国网络安全机构联合发布的 *Careful adoption of agentic AI services* 指出，Agentic AI 已进入关键基础设施、国防和任务关键场景；Agent 可以自动执行重复任务，但也可能导致生产力损失、服务中断、隐私泄露和网络安全事件。该报告强调组织需要持续可见性、保证机制、最小权限、隔离、分阶段部署和持续评估。

对应用户痛点：

- 安全团队不是不知道 Agent 有风险，而是不知道 **Agent 具体做了什么**。
- 开发团队不是不想使用 Agent，而是担心“一开权限就不可控，一管太死又不能提效”。
- 领导/合规团队需要能解释的证据链，而不是“模型可能做错事”的抽象判断。

来源：

- Five Eyes / CISA / NSA / NCSC 等联合指南：
https://media.defense.gov/2026/Apr/30/2003922823/-1/-1/0/CAREFULADOPTINGAGENTICAISERVICES_FINAL.PDF

1.2 OWASP 把“过度代理能力”归因为功能、权限和自治过度

OWASP LLM06:2025 Excessive Agency 将 Agent 风险归纳为 excessive functionality、excessive permissions、excessive autonomy：Agent 工具过多、权

限过大、关键动作缺少人工确认，都会把提示注入、幻觉或工具误用放大为真实破坏。OWASP 还明确建议减少工具、缩小工具功能、避免开放式 shell/fetch URL 等工具、最小化权限、要求人工审批，并在下游系统执行完整授权校验。

对应用户痛点：

- “只读工具”实际可写、可删、可联网，用户难以确认工具边界。
- AI coding 工具最常见的危险能力不是回答，而是 **shell、文件系统、网络、插件/skills**。
- 用户希望保留 Agent 效率，但需要像安全带一样的 runtime guardrail。

来源：

- OWASP LLM06:2025 Excessive Agency: <https://genai.owasp.org/llmrisk/llm062025-excessive-agency/>
- OWASP Agentic AI Threats and Mitigations: <https://genai.owasp.org/resource/agentic-ai-threats-and-mitigations/>

1.3 AI 编码和 Agent Skills 已经把“开发者本机”变成新攻击面

GitGuardian 2026 State of Secrets Sprawl 报告显示，2025 年公开 GitHub 提交中新检测到 28,649,024 个 secrets，AI-service secrets 同比增长 81%，AI-assisted commits 中泄密约为普通基线的 2 倍；MCP 配置文件也暴露了大量 API key、数据库连接串和搜索/AI 服务密钥。Snyk 对 Agent Skills 生态的研究显示，Agent skills 可以继承 shell、文件系统、环境变量/配置凭据和消息通道权限，且大量 skills 存在关键安全问题或恶意 payload。

对应用户痛点：

- 开发者电脑上有 .env、云凭证、SSH key、npm/pip token、MCP 配置，Agent 一旦被提示注入或供应链投毒，会把本机变成外泄跳板。
- 企业扫描代码仓库不够，因为 secrets 还可能存在协作工具、配置文件、开发者工作站和 Agent 工具链里。

- 用户真正要解决的是“哪个 Agent 在哪个任务里读了什么秘密、接着连向哪里、是否应该阻断”。

来源：

- GitGuardian State of Secrets Sprawl 2026: <https://www.gitguardian.com/state-of-secrets-sprawl-report-2026>
- Snyk Clinejection / AI Agent supply-chain attack: <https://snyk.io/blog/cline-supply-chain-attack-prompt-injection-github-actions/>
- Snyk ToxicSkills: <https://snyk.io/blog/toxicskills-malicious-ai-agent-skills-clawhub/>

1.4 NIST 关注跨生命周期治理，而本项目提供运行时证据层

NIST AI 600-1 是 AI RMF 面向生成式 AI 的跨行业 profile，强调组织需要把可信、负责和风险管理纳入 AI 产品全生命周期。对本项目而言，NIST 给出的是治理框架，本项目补的是落地证据层：把 Agent 的进程、文件、网络和策略行为转化为可度量、可回放、可审计的事实。

来源：

- NIST AI 600-1: <https://www.nist.gov/publications/artificial-intelligence-risk-management-framework-generative-artificial-intelligence>

2. 用户痛点画像

用户	真实担忧	本项目切入话术
AI coding 开发者	Agent 很好用，但不知道它是否越权读 .env、跑了危险命令、偷偷外联。	“不禁止你用 Agent，而是给每次 Agent 执行装黑匣子和刹车。”
企业安全团队	EDR 看得到进程，但不知道是不是某个 Agent task 触发；Prompt 安全看不到 OS 行为。	“把 tool_call、PID、文件、网络和策略决策串成一条证据链。”

用户	真实担忧	本项目切入话术
研发效能/平台团队	希望推广 Codex/Claude/Gemini/Copilot，但怕合规和事故追责。	“先观察、再告警、再阻断，支持灰度策略和审计报告。”
高校/竞赛评委	希望看到硬技术，不想只看大屏和概念。	“BPF LSM/cgroup 在内核决策点返回 EACCES 或拒绝 connect，并有 smoke 脚本验证。”
实验室/课程	eBPF 难教，Agent 安全又太抽象。	“用低代码策略和执行图谱把内核安全实验变成可演示项目。”

3. 本仓库代码实现证据

能力	代码证据	对应用户痛点
BPF LSM 同步阻断	backend/ebpf/ lsm_enforcer.c: SEC("lsm/ bprm_check_security")、 SEC("lsm/file_open")、 file_permission、 mmap_file、 file_mprotect、inode_*， 命中 map 后返回 -EACCES。	让 “禁止执行 nc / 禁止读取私钥 / 禁止重命名敏感文件” 在行为完成前失败，而不是事后告警。
cgroup 网络阻断	backend/ebpf/ cgroup_sandbox.c: SEC("cgroup/connect4")、 connect6、sendmsg4、	Agent 或子进程异常外联时，可以按 PID cgroup、IP 或端口在内核层拒绝。

能力	代码证据	对应用户痛点
进程树归因	sendmsg6, 支持 cgroup、IPv4/IPv6、IPv4-mapped IPv6、端口阻断。 backend/event_context.go: processContextStore 保存 agent_run_id/task_id/tool_call_id/trace_id, 子进程可从父 PID 或 cgroup 继承上下文。	解决 Agent 通过 shell/python/node/git/npm/curl 执行时“原始 Agent 丢失”的问题。
统一证据模型	backend/event_envelope.go: 将 wrapper、native_hook、mcp、process、network、exec、file、policy 事件归一为 EventEnvelope。	便于审计、OTLP/MCP 下游集成和竞赛演示, 不只是前端表格。
语义-事实一致性	backend/semantic_alerts.go: 识别 SECRET_ACCESS、WORKSPACE_ESCAPE、UNEXPECTED_NETWORK_EGRESS、UNEXPECTED_CHILD_PROCESS、TOKEN_EXFIL_RISK、MULTI_AGENT_FILE_CONTENTION 等。	把“只读任务却读密钥/外联/启动危险进程”的偏离行为讲清楚。
TLS Agent 元数据	backend/tls_agent_stream.go: 仅提	兼顾应用层语义关联与隐私

能力	代码证据	对应用户痛点
前端安全策略 UI	取 prompt/response 摘要、长度、role、vendor，原文明文保留在显式启用的有界内存 store。 frontend/src/components/config/ ConfigSecurityTab.vue 展示 cgroup/BPF LSM attach 状态、maps、active blocks、decision counters，并提供 block/unblock 控件。	/合规，避免默认采集敏感明文。 安全团队能看见策略是否真的生效，而不是只相信配置文件。
低代码策略生成	frontend/src/components/plugins/recipes.ts 与 transpiler.ts 支持 nc 阻断、SSH 私钥读取保护、敏感 rename 审计等 recipe 并生成 LSM 程序。	让非内核开发者能理解和演示 eBPF 策略，降低教学和试点门槛。
验证脚本	scripts/os-enforcement-preflight.sh 与 scripts/os-enforcement-smoke.sh 检查对象 section、bpffs、BPF LSM、cgroup v2，并验证 LSM/cgroup deny。	回答评委“你怎么证明不是 PPT 能力”的问题。

4. 材料应强化的表达

4.1 痛点表达从“AI 安全泛化”改为“三个不可见”

1. **Agent 意图不可见**：安全日志不知道哪个 tool call 触发。
2. **子进程链路不可见**：真正危险行为在 shell/python/node/curl/npm 中发生。
3. **阻断是否生效不可见**：普通日志只能说发生了，不能证明内核已拒绝。

4.2 方案表达从“可视化平台”改为“三层闭环”

1. **看得见**：eBPF + EventEnvelope 记录进程/文件/网络/策略事实。
2. **说得清**：Agent run/task/tool 与 PID/PPID/cgroup 关联成执行图谱。
3. **拦得住**：BPF LSM/cgroup/wrapper 在高风险动作完成前阻断。

4.3 商业表达从“卖安全工具”改为“Agent 落地安全保险”

企业真正要的是：不阻止研发团队用 Agent，但能回答四个问题：

- 它有没有越权读秘密？
- 它有没有异常外联？
- 它是不是从只读任务漂移到了执行命令？
- 出事后能不能复盘、归因、证明已控制？

5. 申报书/PPT 建议新增演示链路

1. 启动 Agent 任务：声明为“只读代码审查”。
2. Agent/子进程触发敏感行为：读取 .env 或 id_rsa。
3. Dashboard 出现 SECRET_ACCESS 与 SEMANTIC_MISMATCH。
4. Execution Graph 展示 Agent Run → Tool Call → shell/python/curl → File/Network。
5. Security Policy 一键添加 BPF LSM/cgroup block。
6. 再次执行时返回 EACCES 或 connect 失败。

这条链路能同时击中用户痛点、评审创新点和代码实现证据。